Лабораторная работа №4

Выполнил: Казаков А.Е.

Группа: 4404

Задание №1. Используя функциональную декомпозицию, переписать задачу 5 для вычисления значения по формуле (Σx + Σy + Σz) / 3N, где N — количество точек. Использовать глобальную сумму и критические секции.

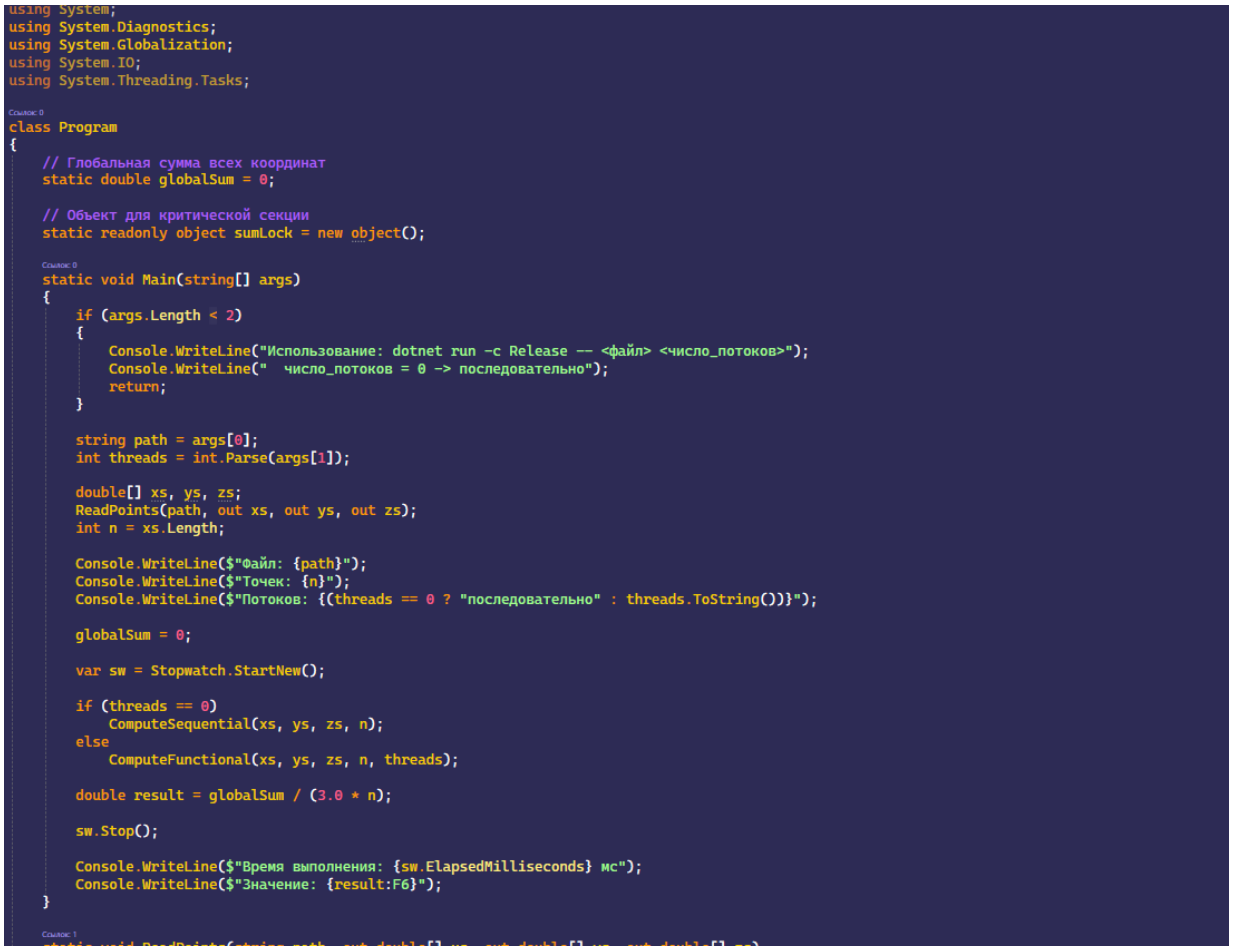
Краткое описание решения.

Программа читает из файла координаты точек в 3D-пространстве (x, y, z), суммирует все координаты и делит полученную глобальную сумму на 3 * N. Результат — одно число, представляющее среднее арифметическое всех координат. Реализованы две версии. Последовательная версия складывает все координаты в один цикл в глобальную переменную globalSum. Параллельная версия использует функциональную декомпозицию: три независимых блока, каждый суммирует координаты по своей оси (x, y, z) в локальную переменную, а затем под критической секцией (lock) один раз добавляет её к глобальной сумме. Критическая секция нужна, поскольку все три потока пишут в одну и ту же переменную globalSum — без синхронизации возникла бы гонка данных, и результат был бы неверным. Локальное накопление с последующим однократным входом в критическую секцию выбрано для того, чтобы избежать захвата блокировки на каждой итерации цикла и снизить накладные расходы.

Решение.

Исходный код программы в двух скриншотах.

Подключение пространств имён, объявление класса Program с глобальной переменной globalSum и объектом sumLock для критической секции, метод Main с разбором аргументов и ветвлением, метод чтения точек из файла, последовательная версия и функциональная декомпозиция через Parallel.Invoke.



```
using System;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Threading.Tasks;

class Program
{
    // Глобальная сумма всех координат
    static double globalSum = 0;

    // Объект для критической секции
    static readonly object sumLock = new object();

    static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.WriteLine("Использование: dotnet run -c Release -- <файл> <число_потоков>");
            Console.WriteLine("  число_потоков = 0 -> последовательно");
            return;
        }

        string path = args[0];
        int threads = int.Parse(args[1]);

        double[] xs, ys, zs;
        ReadPoints(path, out xs, out ys, out zs);
        int n = xs.Length;

        Console.WriteLine($"Файл: {path}");
        Console.WriteLine($"Точек: {n}");
        Console.WriteLine($"Потоков: {(threads == 0 ? "последовательно" : threads.ToString())}");

        globalSum = 0;

        var sw = Stopwatch.StartNew();

        if (threads == 0)
            ComputeSequential(xs, ys, zs, n);
        else
            ComputeFunctional(xs, ys, zs, n, threads);

        double result = globalSum / (3.0 * n);

        sw.Stop();

        Console.WriteLine($"Время выполнения: {sw.ElapsedMilliseconds} мс");
        Console.WriteLine($"Значение: {result:F6}");
    }
```

```
static void ReadPoints(string path, out double[] xs, out double[] ys, out double[] zs)
{
    var lines = File.ReadAllLines(path);
    xs = new double[lines.Length];
    ys = new double[lines.Length];
    zs = new double[lines.Length];

    for (int i = 0; i < lines.Length; i++)
    {
        var p = lines[i].Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
        xs[i] = double.Parse(p[0], CultureInfo.InvariantCulture);
        ys[i] = double.Parse(p[1], CultureInfo.InvariantCulture);
        zs[i] = double.Parse(p[2], CultureInfo.InvariantCulture);
    }
}

// Последовательная версия
static void ComputeSequential(double[] xs, double[] ys, double[] zs, int n)
{
    for (int i = 0; i < n; i++)
    {
        globalSum += xs[i];
        globalSum += ys[i];
        globalSum += zs[i];
    }
}

// Функциональная декомпозиция с критическими секциями
static void ComputeFunctional(double[] xs, double[] ys, double[] zs, int n, int threads)
{
    var options = new ParallelOptions { MaxDegreeOfParallelism = threads };

    Parallel.Invoke(options,
        () =>
        {
            double localSum = 0;
            for (int i = 0; i < n; i++) localSum += xs[i];
            lock (sumLock) { globalSum += localSum; }
        },
        () =>
        {
            double localSum = 0;
            for (int i = 0; i < n; i++) localSum += ys[i];
            lock (sumLock) { globalSum += localSum; }
        },
        () =>
        {
            double localSum = 0;
            for (int i = 0; i < n; i++) localSum += zs[i];
            lock (sumLock) { globalSum += localSum; }
```

Результаты запуска программы.

Последовательные запуски dotnet run -c Release -- points.txt N для N = 0, 2, 3, 8. Файл из 1 000 000 точек. Значение (Σx + Σy + Σz) / 3N одинаково во всех запусках и равно 49999072,309130 — это подтверждает корректность критической секции. Параллельная версия работает медленнее последовательной из-за накладных расходов на создание задач и синхронизацию, которые на задаче такого размера превышают сам объём полезной работы.

```
PS C:\Users\0\Desktop\3123123123\lab4> dotnet run -c Release -- points.txt 0
Файл: points.txt
Точек: 1000000
Потоков: последовательно
Время выполнения: 4 мс
Значение: 49999072,309130
PS C:\Users\0\Desktop\3123123123\lab4> dotnet run -c Release -- points.txt 2
Файл: points.txt
Точек: 1000000
Потоков: 2
Время выполнения: 20 мс
Значение: 49999072,309130
PS C:\Users\0\Desktop\3123123123\lab4> dotnet run -c Release -- points.txt 3
Файл: points.txt
Точек: 1000000
Потоков: 3
Время выполнения: 14 мс
Значение: 49999072,309130
PS C:\Users\0\Desktop\3123123123\lab4> dotnet run -c Release -- points.txt 8
Файл: points.txt
Точек: 1000000
Потоков: 8
Время выполнения: 14 мс
Значение: 49999072,309130
```

Сводная таблица результатов

Потоки	Время, мс	Ускорение
0	4	1,00
2	20	0,20
3	14	0,29
8	14	0,29

Значение (Σx + Σy + Σz) / 3N одинаково во всех запусках и равно **49999072,309130**. Как видно из таблицы, параллельные версии медленнее последовательной, то есть ускорения на задаче такого размера получить не удалось.

Вывод. В ходе работы я переписал программу из задачи 5 так, чтобы она вычисляла единое значение (Σx + Σy + Σz) / 3N с использованием глобальной суммы и критических секций. Функциональная декомпозиция реализована через Parallel.Invoke с тремя независимыми блоками, каждый из которых суммирует координаты по своей оси в локальную переменную и затем один раз добавляет её в глобальную сумму под lock. Значение получилось одинаковым во всех запусках и равным 49999072,309130, что подтверждает корректность синхронизации. Ускорения при этом получить не удалось: на 1 000 000 точек параллельная версия оказалась в 3–5 раз медленнее последовательной. Это объясняется тем, что задача слишком мала — накладные расходы на создание задач и синхронизацию превышают сам объём полезной работы, а обращение к трём разным массивам из разных потоков ухудшает эффективность кэша. Число потоков свыше трёх не даёт никакого эффекта, поскольку независимых блоков ровно три. Это показывает, что функциональная декомпозиция имеет смысл только тогда, когда задача достаточно велика, чтобы окупить накладные расходы, и число независимых частей велико.

Ответы на вопросы

Зачем нужна критическая секция в этой задаче?

При функциональной декомпозиции три блока работают параллельно, и в конце каждый добавляет свою локальную сумму в общую переменную globalSum. Если бы этой операции не предшествовала критическая секция, все три потока могли бы одновременно читать текущее значение globalSum, прибавлять к нему своё и записывать результат обратно. Из-за чередования этих операций часть прибавленных значений была бы потеряна — возникла бы гонка данных, и итоговый результат оказался бы меньше истинного. Критическая секция гарантирует, что в каждый момент времени только один поток изменяет globalSum, поэтому результат получается детерминированным и правильным.

Почему локальная сумма накапливается до критической секции, а не блокируется на каждой итерации?

Если бы каждый поток захватывал блокировку на каждой итерации цикла, время выполнения выросло бы в разы из-за накладных расходов на синхронизацию. Гораздо эффективнее сначала просуммировать все свои значения в локальной переменной без синхронизации (это чистая работа процессора над своими данными), а затем один раз зайти в критическую секцию и добавить всю сумму целиком.

Почему параллельная версия оказалась медленнее последовательной?

Причина в том, что задача слишком мала для того, чтобы параллелизм дал выигрыш. Три миллиона операций сложения типа double современный процессор выполняет примерно за 4 миллисекунды, при этом сам работает с векторными инструкциями и эффективно использует кэш. Накладные расходы на создание пула потоков, распределение трёх задач и синхронизацию через lock занимают порядка 10–20 миллисекунд — это больше, чем время выполнения самой полезной работы. Кроме того, каждый из трёх блоков читает свой отдельный массив (xs, ys или zs), что ухудшает попадание в кэш по сравнению с последовательной версией, где три массива обходятся синхронно по одному индексу i. В сумме это приводит к тому, что параллельная версия оказывается в 3–5 раз медленнее.

Почему ускорение не растёт после трёх потоков?

Функциональная декомпозиция разбивает работу на три независимых блока: суммирование по x, по y и по z. Даже если задать в ParallelOptions больше потоков, задач всё равно три, и Parallel.Invoke распределит их только по трём потокам, а остальные будут простаивать. Это подтверждается таблицей: с 3 и с 8 потоками время одинаковое — 14 мс.